

Mảng, chuỗi và hai con trỏ (Arrays, Strings, Two Pointers)

PHỤ THUỘC A/03 cho ngân sách chỉ phí và phân cấp bộ nhớ; A/04 cho cách viết bất biến — chương này là nơi bất biến trả công nhiều nhất.

TÓM TẮT

Mảng là cấu trúc dữ liệu bị đánh giá thấp nhất: nó không có “thuật toán” nào hào nhoáng, nhưng nó thắng gần như mọi cấu trúc khác về hằng số và về tính thân thiện với cache. Chương này đi qua bốn kỹ thuật biến $O(n^2)$ thành $O(n)$ mà không cần thêm cấu trúc nào: tổng tiền tố, mảng hiệu, hai con trỏ, và cửa sổ trượt. Điểm chung của cả bốn là một ý duy nhất — *đừng tính lại từ đầu, hãy cập nhật phần thay đổi*. Nếu chỉ nhớ một điều: khi thấy hai vòng lặp lồng nhau, hãy hỏi “khi vòng ngoài tiến một bước, vòng trong có thật sự phải quay lại từ đầu không”; câu trả lời thường là không, và đó là toàn bộ chương này.

KIẾN THỨC CẦN CÓ TRƯỚC

- **mảng (array)** — dãy phần tử cùng kiểu nằm liên tiếp trong bộ nhớ; truy cập phần tử thứ i tốn $O(1)$ bằng phép tính địa chỉ.
- **tổng tiền tố (prefix sum)** — mảng P với $P_i = a_1 + \dots + a_i$; cho phép lấy tổng đoạn $[l, r]$ bằng $P_r - P_{l-1}$ trong $O(1)$.
- **mảng hiệu (difference array)** — mảng D với $D_i = a_i - a_{i-1}$; cho phép cộng một hằng số vào cả đoạn trong $O(1)$, rồi khôi phục mảng gốc bằng tổng tiền tố.
- **hai con trỏ (two pointers)** — kỹ thuật duy trì hai chỉ số cùng quét qua dữ liệu, mỗi chỉ số chỉ đi tiến, cho tổng chỉ phí $O(n)$ thay vì $O(n^2)$.
- **cửa sổ trượt (sliding window)** — trường hợp riêng của hai con trỏ: duy trì một đoạn liên tiếp thoả điều kiện, mở rộng bên phải và co bên trái.
- **tính đơn điệu** — tính chất “nếu đoạn $[l, r]$ thoả thì mọi đoạn con của nó cũng thoả” (hoặc chiều ngược lại). Đây là điều kiện sống còn để hai con trỏ đúng.

CÂU HỎI MỞ ĐẦU

- Q1. Vì sao mảng — cấu trúc thô sơ nhất — lại thường thắng những cấu trúc tinh vi hơn trên máy thật, dù độ phức tạp tiệm cận tệ hơn?
- Q2. Hai con trỏ chạy $O(n)$ dù trông như hai vòng lặp lồng nhau. Lập luận nào cho ta biết nó thật sự tuyến tính?
- Q3. Kỹ thuật hai con trỏ hỏng khi nào? Cho một bài mà áp dụng nó sẽ cho kết quả sai.
- Q4. Tổng tiền tố và mảng hiệu là hai phép toán ngược nhau. Việc nhận ra điều đó cho bạn thêm kỹ thuật gì?
- Q5. Vì sao “sắp xếp trước” lại là nước đi mở đầu hiệu quả đến vậy, dù bản thân nó tốn $O(n \log n)$?

Có một hiện tượng đáng chú ý trong lập trình thi đấu: rất nhiều bài được gắn nhãn khó lại có lời giải chỉ gồm một vòng lặp và hai biến chỉ số. Không cấu trúc dữ liệu, không đệ quy, không công

thức. Người chưa quen nhìn lời giải đó thấy nó như trò ảo thuật — rõ ràng bài đòi xét mọi đoạn con, mà đây chỉ quét một lượt.

Không có ảo thuật nào. Cả chương này chỉ khai thác một quan sát duy nhất, chính là câu hỏi số 10 trong danh sách heuristic ở chương A/01: *cái gì thực sự thay đổi giữa hai bước liên tiếp?* Nếu đáp án của bước $i + 1$ chỉ khác đáp án của bước i một chút, thì tính lại từ đầu là lãng phí, và toàn bộ chi phí sụp từ $O(n^2)$ xuống $O(n)$. Bốn kỹ thuật trong chương — tổng tiền tố, mảng hiệu, hai con trỏ, cửa sổ trượt — là bốn cách hiện thực hoá quan sát đó cho bốn tình huống khác nhau.

1.1 Vì sao bắt đầu bằng mảng

Trước khi vào kỹ thuật, cần một lời biện hộ cho mảng, vì người mới thường vội bỏ qua nó để đi học cấu trúc “xịn” hơn.

Mảng có ba ưu thế mà không cấu trúc nào lấy lại được. Thứ nhất, truy cập phần tử là một phép tính địa chỉ, không có con trỏ để lần theo. Thứ hai, các phần tử nằm liên tiếp trong bộ nhớ, nên khi bạn đọc phần tử i thì phần cứng đã kéo sẵn cả một khối chứa $i + 1, i + 2, \dots$ vào cache — duyệt tuần tự gần như miễn phí sau lần đọc đầu. Thứ ba, không có chi phí cấp phát cho từng phần tử.

Hệ quả thực hành mạnh hơn nhiều người nghĩ, và nó nổi thẳng với hình phân cấp bộ nhớ ở chương A/03: một thuật toán $O(n \log n)$ nhảy ngẫu nhiên khắp bộ nhớ có thể thua một thuật toán $O(n^2)$ quét tuần tự, ở những kích thước n rất thực tế. Đây là lý do các cấu trúc dữ liệu “cây” trong thực chiến thường được cài bằng mảng phẳng thay vì bằng con trỏ (B/10), và là lý do khi tối ưu hằng số thì việc đầu tiên nên thử là đổi cấu trúc có con trỏ sang mảng.

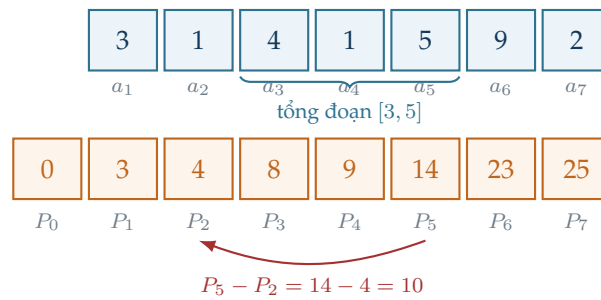
CẠM BÃY

Cạm bẫy ngược lại cũng có thật: mảng đắt ở phép chèn và xoá giữa mảng, vì phải dịch chuyển toàn bộ phần đuôi. Nếu bài của bạn chèn và xoá liên tục ở giữa, mảng là lựa chọn sai và bạn cần cấu trúc ở chương B/09 hoặc B/10. Quy tắc: mảng thắng khi *cấu trúc dữ liệu tĩnh, truy vấn nhiều*; nó thua khi *cấu trúc thay đổi liên tục ở giữa*.

1.2 Tổng tiền tố: trả trước một lượt quét, sau đó mọi truy vấn miễn phí

Bài toán mở đầu: cho mảng và q truy vấn, mỗi truy vấn hỏi tổng của một đoạn $[l, r]$. Cách ngây thơ tốn $O(n)$ mỗi truy vấn, tổng $O(nq)$. Với $n = q = 10^5$ thì đó là 10^{10} phép toán — quá xa ngân sách ở chương A/03.

Ý tưởng: tính trước mảng P với $P_0 = 0$ và $P_i = P_{i-1} + a_i$. Khi đó tổng đoạn $[l, r]$ bằng $P_r - P_{l-1}$. Chi phí: $O(n)$ một lần, rồi $O(1)$ mỗi truy vấn.



Hình 1.1. Tổng tiền tố. Hàng trên là mảng gốc, hàng dưới là tổng tiền tố với một ô đệm $P_0 = 0$ — ô đệm này loại bỏ mọi trường hợp riêng khi $l = 1$, và bỏ nó đi là nguồn lỗi lệch một đơn vị phổ biến nhất của kỹ thuật này.

Ý tưởng mở rộng theo ba hướng và cả ba đều hay gặp. *Hai chiều*: với ma trận, $P_{i,j}$ là tổng của hình chữ nhật từ góc trên trái tới (i, j) ; tổng của một hình chữ nhật bất kỳ tính bằng bốn giá trị theo nguyên lý bao hàm–loại trừ ($A/05$). Trong thị giác máy tính, cấu trúc này có tên riêng là *ảnh tích phân* và nó là thứ khiến bộ phát hiện khuôn mặt Viola–Jones chạy được thời gian thực năm 2001. *Phép toán khác tổng*: XOR dùng được vì có phép ngược; tích thì phải cẩn thận với số 0; còn cực đại thì không dùng được vì không có phép ngược — đó chính là lúc bạn cần sparse table hoặc cây phân đoạn ($B/10$). *Tổng tiền tố theo điều kiện*: đếm số phần tử thoả tính chất nào đó trong đoạn, bằng cách xây tổng tiền tố trên mảng chỉ báo 0/1.

Một mẹo hệ quả đáng nhớ: bài “đếm số đoạn con có tổng bằng k ” giải bằng tổng tiền tố cộng bảng băm — vì tổng đoạn $[l, r] = k$ tương đương $P_r - P_{l-1} = k$, tức là với mỗi r ta chỉ cần đếm xem có bao nhiêu P trước đó bằng $P_r - k$. Cách nghĩ này — *biến điều kiện trên đoạn thành điều kiện trên hai điểm đầu mút* — là một mẫu dùng lại rất nhiều lần.

1.3 Mảng hiệu: phép ngược của tổng tiền tố

Nếu tổng tiền tố giải bài “cập nhật ít, truy vấn đoạn nhiều”, thì mảng hiệu giải bài đối xứng: “cập nhật đoạn nhiều, đọc kết quả một lần ở cuối”.

Cách làm: để cộng v vào mọi phần tử trong đoạn $[l, r]$, ta chỉ cần $D_l += v$ và $D_{r+1} -= v$. Mỗi cập nhật tốn $O(1)$ thay vì $O(r - l + 1)$. Cuối cùng, lấy tổng tiền tố của D để khôi phục mảng thật.

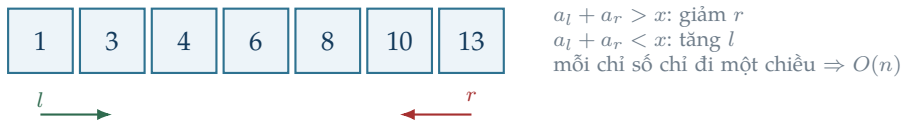
Nhận ra hai kỹ thuật này là nghịch đảo của nhau cho bạn một mẫu tư duy tổng quát: nếu một phép toán tốn kém khi làm trực tiếp, hãy tìm một biểu diễn khác của cùng dữ liệu mà trong đó phép toán ấy rẻ, rồi chuyển đổi một lần ở cuối. Cùng ý tưởng xuất hiện lại ở phép biến đổi Fourier ($D/21$), nơi phép nhân chập tốn $O(n^2)$ trong miền thời gian trở thành phép nhân từng điểm $O(n)$ trong miền tần số.

1.4 Hai con trỏ: vì sao hai vòng lặp lồng nhau lại tuyến tính

Bài toán mẫu: mảng đã sắp tăng, tìm cặp có tổng bằng x . Vét cạn xét mọi cặp, $O(n^2)$. Kỹ thuật hai con trỏ: đặt l ở đầu, r ở cuối. Nếu $a_l + a_r > x$ thì giảm r ; nếu nhỏ hơn thì tăng l ; nếu bằng thì tìm thấy.

Vì sao đúng? Bất biến là mẫu chốt, và đây là một bất biến đẹp: mọi cặp đã bị loại đều chắc chắn không phải đáp án. Khi $a_l + a_r > x$, mọi cặp gồm a_r với một phần tử ở vị trí $\geq l$ đều có tổng $\geq a_l + a_r > x$, nên loại r là hợp lệ. Vì sao tuyến tính? Vì l chỉ tăng và r chỉ giảm, mỗi chỉ số đi tối đa n bước, tổng

cộng tối đa $2n$ bước — lập luận này được gọi là *tính chi phí bằng tổng quãng đường của con trỏ*, và nó dùng lại cho mọi biến thể.



Hình 1.2. Hai con trỏ trên mảng đã sắp. Điều kiện sống còn là *tính đơn điệu*: khi r giảm thì tổng chỉ có thể giảm, khi l tăng thì tổng chỉ có thể tăng. Mất tính đơn điệu — ví dụ khi mảng chưa sắp, hoặc khi có số âm trong bài cửa sổ trượt về tổng — thì kỹ thuật này cho kết quả sai chứ không phải chậm.

Biến thể quan trọng nhất là *cửa sổ trượt*: tìm đoạn liên tiếp dài nhất (hoặc ngắn nhất) thỏa một điều kiện. Khuôn chung: mở rộng r từng bước; sau mỗi lần mở rộng, nếu điều kiện bị vi phạm thì co l cho tới khi hợp lệ trở lại; cập nhật đáp án. Điều kiện để khuôn này đúng là tính đơn điệu theo nghĩa: nếu đoạn $[l, r]$ vi phạm thì mọi đoạn chứa nó cũng vi phạm.

Đây chính là chỗ hỏng thường gặp. Bài “đoạn con dài nhất có tổng $\leq S$ ” giải được bằng cửa sổ trượt nếu mọi số đều không âm; có số âm thì tổng không còn đơn điệu theo độ dài, và cửa sổ trượt cho đáp án sai. Bài đó phải chuyển sang tổng tiền tố cộng cấu trúc tìm kiếm. Khi gặp một bài cửa sổ trượt, câu hỏi đầu tiên luôn phải là: *tính chất này có đơn điệu không?*

CẦN NHỚ

Ba câu hỏi trước khi dùng hai con trỏ: (i) dữ liệu có thứ tự cần thiết chưa — nếu chưa, có sắp được không; (ii) tính chất có đơn điệu không — có lại thì luôn “dễ thỏa hơn”, mở ra thì luôn “khó thỏa hơn”; (iii) mỗi con trỏ có chỉ đi một chiều không. Thiếu bất kỳ điều nào thì kết quả sai, và cái sai đó *im lặng* — chương trình vẫn chạy, chỉ ra số khác.

1.5 Sắp xếp như một nước đi mở đầu

Một mẫu xuất hiện liên tục: rất nhiều bài trở nên dễ sau khi sắp xếp. Điều này đáng được nói thành nguyên tắc, vì nó là heuristic có tỉ lệ thành công cao và chi phí thấp — $O(n \log n)$ gần như luôn nằm trong ngân sách.

Sắp xếp có tác dụng vì nó tạo ra thứ mà ba kỹ thuật trên cần: *tính đơn điệu*. Sau khi sắp, hai con trỏ dùng được, tìm kiếm nhị phân dùng được, tham lam thường lộ ra, và các phần tử giống nhau nằm cạnh nhau nên đếm bội số thành chuyện tầm thường.

Một biến thể đáng nhớ là *sắp xếp theo tiêu chí do bạn thiết kế*. Ví dụ với các đoạn thẳng, sắp theo điểm kết thúc cho bài xếp lịch tham lam (C/15); sắp theo điểm bắt đầu cho bài gộp đoạn giao nhau; sắp theo hiệu của hai đại lượng cho các bài đánh đổi. Việc chọn khóa sắp xếp thường chính là toàn bộ lời giải, và cách kiểm tra nó đúng là lập luận đổi chỗ ở chương A/04.

1.6 Chuỗi: mảng có tính chất riêng

Chuỗi là mảng ký tự, nên mọi kỹ thuật trên đều áp dụng được, nhưng có ba điểm riêng đáng nhớ ngay từ đây.

Thứ nhất, *so sánh hai chuỗi không phải $O(1)$ mà là $O(\text{độ dài})$* — điều mà chương A/03 đã cảnh báo. Dùng chuỗi làm khóa của cấu trúc tìm kiếm thì nhân thêm một thừa số vào độ phức tạp.

Thứ hai, bảng chữ cái thường nhỏ, và điều đó mở ra kỹ thuật đếm bằng mảng cố định 26 hoặc 128 phần tử thay vì bảng băm — nhanh hơn nhiều lần. Rất nhiều bài chuỗi trong phỏng vấn (đếm anagram, cửa sổ chứa đủ ký tự) là bài cửa sổ trượt với một mảng đếm 26 phần tử.

Thứ ba, các bài so khớp mẫu có cấu trúc đặc thù và cần công cụ riêng — hàm tiền tố, hàm Z, băm chuỗi — toàn bộ là nội dung chương B/12.

1.7 Nhìn cửa sổ di chuyển, rồi chủ động làm nó hỏng

Với $a = [2, 1, 3, 2]$ và $S = 4$, ta tìm đoạn dài nhất có tổng không vượt S . Hình 1.3 cho thấy vì sao một bước mở rộng có thể kéo theo nhiều bước co. Chỉ sau khi co xong mới cập nhật đáp án. Tổng số bước co vẫn không quá n , vì đầu trái không quay lại.

[1, 1]	2	1	3	2	tổng = 2	
[1, 2]	2	1	3	2	tổng = 3	
[1, 3]	2	1	3	2	tổng = 6	vi phạm: bỏ 2 rồi bỏ 1
[3, 3]	2	1	3	2	tổng = 3	
[3, 4]	2	1	3	2	tổng = 5	vi phạm: bỏ 3
[4, 4]	2	1	3	2	tổng = 2	

Hình 1.3. Vết chạy cửa sổ trượt với chỉ số từ 1. Ô xanh là cửa sổ hiện tại, ô trắng nằm ngoài. Hai hàng tổng 6 và 5 là trạng thái tạm thời chưa hợp lệ; đáp án tốt nhất là độ dài 2 ở đoạn $[1, 2]$.

Để kiểm tra giả thiết, đổi dữ liệu thành $[5, -2]$, vẫn lấy $S = 4$. Khuôn vừa dùng loại 5 ngay ở bước đầu, nên không thể tìm lại đoạn dài 2 có tổng 3. Đây là phản ví dụ đủ nhỏ để nhớ: số âm có thể cứu một cửa sổ từng vi phạm. Đừng sắp xếp để chữa bài này vì sắp xếp phá tính liên tiếp của đoạn gốc.

Bảng 1.1. Chọn kỹ thuật trên mảng theo thao tác cần làm. Chi phí giả sử phép toán trên một giá trị tồn hằng số.

Kỹ thuật	Giả thiết / cơ chế	Chi phí	Chỗ dễ sai
Tổng tiền tố	Mảng tĩnh, lấy hiệu hai tiền tố	Xây $O(n)$, hỏi $O(1)$	Thiếu P_0 ; tràn tổng
Mảng hiệu	Cộng đoạn, đọc kết quả sau cùng	Sửa $O(1)$, khôi phục $O(n)$	Thiếu ô $r + 1$; hỏi giữa chừng
Cửa sổ trượt	Bỏ một đầu nút không mất nghiệm tương lai	Tổng $O(n)$ nếu hai đầu chỉ tiến	Tổng có số âm mất đơn điệu
Tiền tố + băm	Đếm tổng đoạn bằng k , cho phép số âm	$O(n)$ kỳ vọng, bộ nhớ $O(n)$	Quên tần suất tiền tố 0 bằng 1

Bảng 1.1 phân biệt bốn bài toán thường bị nhận nhầm thành cùng một mẫu. Khi đếm tổng bằng k , hãy tra số tiền tố bằng $P_r - k$ trước khi thêm P_r ; đảo thứ tự sẽ đếm cả đoạn rỗng nếu $k = 0$.

1.8 Phản biện, nhược điểm và rủi ro triển khai

Mảng, tổng tiền tố và hai con trỏ là những công cụ cơ bản nhất, nhưng chính sự đơn giản đó tạo ra cảm giác an toàn giả tạo. Trong thực tế, đây là nơi phát sinh nhiều lỗi im lặng và lạm dụng phương pháp nhất.

Phản biện: sự lạm dụng hai con trỏ và giới hạn của mảng hiệu. Hai con trỏ chỉ chạy đúng khi bài toán có tính đơn điệu (monotonicity) rõ ràng: mở rộng khoảng làm điều kiện vi phạm nhiều hơn, và thu hẹp khoảng làm điều kiện dễ thoả mãn hơn. Khi mảng xuất hiện số âm, tổng đoạn con dao động hỗn loạn; việc co/dãn cửa sổ tham lam sẽ bỏ sót nghiệm tối ưu toàn cục. Cố chấp ép hai con trỏ vào mảng có số âm là sai lầm kinh điển của người mới. Tương tự, mảng hiệu chỉ là giải pháp xử lý ngoại tuyến (offline batch): cập nhật toàn bộ rồi đọc kết quả một lần duy nhất. Nếu bài toán có truy vấn xen kẽ cập nhật (online dynamic), mảng hiệu hoàn toàn vô dụng; việc khôi phục mảng sau mỗi truy vấn làm độ phức tạp thoái hoá thành $O(q \cdot n)$, chậm hơn hàng nghìn lần so với cây Fenwick hay Segment Tree ($B/10$).

Nhược điểm cốt lõi về cấu trúc và bộ nhớ:

- *Tính tĩnh tuyệt đối (Rigid Immutability):* Cả tổng tiền tố và mảng hiệu đều phụ thuộc vào chỉ số cố định của mảng. Chỉ cần chen thêm hoặc xoá bớt một phần tử ở đầu mảng, toàn bộ bảng tính trước bị vô hiệu hoá và buộc phải xây lại từ đầu với chi phí $O(n)$.
- *Sự phình to bộ nhớ của tiền tố đa chiều:* Trên mảng hai chiều hoặc ba chiều (như voxel 3D trong thị giác máy tính), bảng tổng tiền tố đòi hỏi bộ nhớ $O(n^d)$. Với ảnh phân giải cao hoặc lưới thể tích 3D, dung lượng RAM bùng nổ nhanh chóng; đồng thời công thức bao hàm-loại trừ (Inclusion-Exclusion) nhiều chiều rất dễ nhầm lẫn dấu tại các góc và gây trượt cache L1 liên tục.

Rủi ro vận hành và các lỗi im lặng nguy hiểm:

- *Tràn số nguyên trong tổng tiền tố (Overflow):* Với $n = 10^5$ phần tử và giá trị phần tử tới 10^9 , tổng tiền tố vượt quá 10^{14} , vượt xa giới hạn số nguyên 32-bit có dấu. Dùng `int` thay vì `long` (hoặc `int64_t`) là lỗi im lặng phổ biến nhất: chương trình chạy mượt mà nhưng trả về kết quả âm vô lý. Trong bài toán lấy modulo, hiệu $P[r] - P[l - 1]$ có thể âm, nếu quên cộng thêm m trước khi lấy dư tiếp theo $((P[r] - P[l - 1] \% m + m) \% m)$, kết quả sẽ bị sai hoàn toàn.
- *Lỗi lệch một đơn vị (Off-by-One) và truy cập ngoài biên:* Quên khởi tạo $P[0] = 0$ làm sai lệch truy vấn đoạn bắt đầu từ đầu mảng $[1, r]$. Trong mảng hiệu, việc cập nhật $D[r+1] -= x$ đòi hỏi mảng hiệu phải có kích thước tối thiểu $n + 2$; quên điều này sẽ ghi đè vào vùng nhớ ngoài biên (Out-of-Bounds Memory Write), làm hỏng dữ liệu của các biến lân cận.
- *Trượt cửa sổ vô hạn hoặc vượt biên:* Trong thuật toán hai con trỏ, nếu vòng lặp co cửa sổ không kiểm tra điều kiện $l \leq r$, con trỏ l có thể vượt qua r , tạo ra cửa sổ có độ dài âm và rơi vào vòng lặp vô tận (hang/infinite loop) trên các trường hợp biên mảng rỗng hoặc toàn phần tử không thoả mãn.

CẠM BÃY

Không bao giờ dùng kiểu `int` 32-bit cho mảng tổng tiền tố trừ khi đề bài bảo đảm rõ ràng tổng toàn mảng nằm gọn trong khoảng $[-2 \cdot 10^9, 2 \cdot 10^9]$. Một thói quen tốt là luôn khai báo mảng tiền tố bằng `long` ngay từ đầu để loại bỏ hoàn toàn một lớp lỗi im lặng.

1.9 Gốc rễ: từ băng từ tới ảnh tích phân

Mảng là cấu trúc dữ liệu cổ nhất trong ngành và lý do rất vật lý: bộ nhớ máy tính *vốn* là một dãy ô đánh số, nên mảng không phải một phát minh mà là hình dạng tự nhiên của phần cứng. Mọi cấu trúc khác trong phần B đều được xây *trên* mảng, và cái giá của chúng là chi phí gián tiếp so với truy cập trực tiếp.

Tổng tiền tố thì cổ hơn máy tính. Ý tưởng “lập bảng tổng lũy tích để tra nhanh” đã có trong các bảng thống kê và bảng thiên văn từ trước; trong toán, đó chính là mối quan hệ giữa tổng và sai phân, tương ứng rời rạc của tích phân và đạo hàm — và mối quan hệ đó giải thích vì sao mảng hiệu là nghịch đảo của tổng tiền tố, đúng như đạo hàm là nghịch đảo của tích phân.

Điều thú vị là kỹ thuật này được phát minh lại nhiều lần ở các ngành khác nhau. Trong tính toán song song, phép *scan* (tổng tiền tố) được nhận ra là một nguyên thủy cơ bản mà rất nhiều thuật toán song song quy về được, và nó song song hoá được trong $O(\log n)$ bước — điều không hiển nhiên chút nào vì định nghĩa của nó có vẻ hoàn toàn tuần tự. Trong thị giác máy tính, cùng cấu trúc mang tên *ảnh tích phân*: Viola và Jones dùng nó năm 2001 để tính tổng cường độ trên hình chữ nhật bất kỳ trong bốn phép truy cập, và chính điều đó biến bộ phát hiện khuôn mặt của họ từ ý tưởng thành sản phẩm chạy thời gian thực trên máy ảnh số.

Hai con trỏ thì không có “người phát minh” — nó thuộc loại kỹ thuật hiển nhiên khi nhìn đúng góc, được dùng rải rác từ lâu, và chỉ được đặt tên và hệ thống hoá khi lập trình thi đấu cần một từ vựng chung để nói về nó.

■ Gốc rễ: ý tưởng này ra đời từ bài toán nào

Mốc	Bài toán thúc ép lúc đó	Điều thay đổi sau đó
Thập niên 1940 — máy tính có bộ nhớ đánh địa chỉ	Cần một cách tổ chức dữ liệu khớp với phần cứng.	Mảng: không phải phát minh mà là hình dạng tự nhiên của bộ nhớ; mọi cấu trúc khác đều xây trên nó.
Trước cả máy tính — bảng lũy tích trong thống kê, thiên văn	Tra nhanh tổng của một khoảng trong bảng số.	Tổng tiền tố như một dạng tích phân rời rạc; mảng hiệu là sai phân, tức phép ngược.
Thập niên 1980–90 — tính toán song song	Nhiều thuật toán song song cần cùng một nguyên thủy tích lũy.	Phép <i>scan</i> song song hoá được trong $O(\log n)$ bước dù định nghĩa trông thuần tuần tự; trở thành khối xây dựng chuẩn của GPU (D/24).
2001 — Viola & Jones	Phát hiện khuôn mặt thời gian thực: cần tổng cường độ trên hàng nghìn hình chữ nhật mỗi khung hình.	Ảnh tích phân — tổng tiền tố hai chiều — cho mỗi hình chữ nhật trong bốn phép truy cập; biến thuật toán từ không khả thi thành sản phẩm thương mại.
Thập niên 2000 — lập trình thi đấu	Cần từ vựng chung để dạy các mẫu lặp lại.	“Hai con trỏ”, “cửa sổ trượt” được đặt tên và hệ thống hoá; kỹ thuật cũ nhưng cái tên mới làm chúng dạy được.

■ 1.10 Liên hệ ngang

■ Liên hệ ngang: ý này còn sống ở đâu nữa

Lĩnh vực	Ý tưởng tương ứng	Giống ở đâu, khác ở đâu
Thị giác máy tính	Ảnh tích phân, bộ lọc hộp, tính đặc trưng Haar	Cùng một cấu trúc với tổng tiền tố hai chiều. Bài học chung: khi cần rất nhiều truy vấn trên vùng chữ nhật, hãy trả trước một lượt quét. Ứng dụng gần: tính tổng cường độ trên các ô lưới khi lọc điểm đặc trưng trong SLAM.
Giải tích	Tích phân và đạo hàm là hai phép ngược nhau	Tổng tiền tố và mảng hiệu là bản rời rạc chính xác của cặp này. Nhận ra sự tương ứng giúp nhớ được cả hai và biết khi nào đổi qua lại.
Xử lý tín hiệu	Trung bình trượt, bộ lọc chạy	Cùng khuôn cửa sổ trượt, chỉ khác là ở đó cửa sổ có độ dài cố định nên còn đơn giản hơn. Cả hai đều dựa vào việc cập nhật gia tăng thay vì tính lại.
Mạng máy tính	Cửa sổ trượt trong TCP	Cùng tên và cùng ý tưởng cơ bản: duy trì một khoảng hợp lệ, tiến mép phải khi nhận được xác nhận, kéo mép trái theo. Khác: ở đó cửa sổ điều tiết tốc độ, không phải để tìm cực trị.
Tính toán song song, GPU	Nguyên thủy <i>scan</i> và <i>reduce</i>	Tổng tiền tố là nguyên thủy song song cơ bản. Điểm bất ngờ đáng nhớ: một phép định nghĩa hoàn toàn tuần tự vẫn có thể song song hoá trong $O(\log n)$ bước bằng cây tổng.
Cơ sở dữ liệu	Chỉ mục tổng hợp, bảng tổng lũy tích	Cùng đánh đổi: trả trước chi phí xây dựng để truy vấn rẻ. Cùng nhược điểm: dữ liệu thay đổi thì phải xây lại — và đó chính là lý do tồn tại của cây chỉ số ở B/10.

TRẢ LỜI CÂU HỎI MỞ ĐẦU

- A1. Vì ba lý do vật lý mà ký hiệu tiệm cận không nhìn thấy: truy cập là phép tính địa chỉ chứ không phải lần theo con trỏ; các phần tử nằm liên tiếp nên phần cứng nạp sẵn phần tử kế tiếp vào cache; và không có chi phí cấp phát cho từng phần tử. Với dữ liệu vừa cỡ vài triệu phần tử, ba yếu tố này cộng lại có thể tạo chênh lệch cả chục lần — đủ để lật ngược một khác biệt tiệm cận nhỏ như thừa số log.
- A2. Lập luận “tổng quãng đường của con trỏ”. Vòng lặp ngoài chỉ tăng l , vòng lặp trong chỉ giảm r , và không chỉ số nào quay lui. Mỗi chỉ số vì thế đi tối đa n bước trong suốt cả thuật toán, nên tổng số bước tối đa là $2n$. Cấu trúc lồng nhau của code đánh lừa: cái quyết định chi phí không phải độ sâu lồng nhau mà là tổng số lần các biến chỉ số thay đổi.
- A3. Nó hỏng khi mất tính đơn điệu. Ví dụ: tìm đoạn con dài nhất có tổng $\leq S$ trong mảng có số âm. Với mảng $[5, -6, 5]$ và $S = 4$: cửa sổ trượt sẽ co bỏ phần tử đầu ngay khi tổng vượt S và bỏ sót đoạn $[5, -6, 5]$ có tổng 4. Nguyên nhân: khi có số âm, việc mở rộng cửa sổ có thể làm tổng giảm, nên điều kiện không còn đơn điệu theo độ dài. Bài đó phải giải bằng tổng tiền tố cộng cấu trúc tìm kiếm.
- A4. Nó cho một mẫu tổng quát: nếu phép toán A rẻ trên biểu diễn gốc còn phép toán B đắt, hãy tìm biểu diễn mà trong đó B rẻ, làm hết các thao tác B rồi chuyển đổi ngược một lần. Cụ thể: tổng tiền tố cho “truy vấn đoạn rẻ, cập nhật đắt”; mảng hiệu cho “cập nhật đoạn rẻ, đọc kết quả một lần”. Khi bài cần cả hai đều nhiều thì không biểu diễn tĩnh nào đủ, và đó chính là lúc cần cây chỉ số ở B/10.
- A5. Vì sắp xếp tạo ra tính đơn điệu, và tính đơn điệu là điều kiện đầu vào của hầu hết kỹ thuật rẻ: hai con trỏ, tìm kiếm nhị phân, tham lam, gộp các phần tử bằng nhau. Chi phí $O(n \log n)$ gần như luôn nằm trong ngân sách, nên đây là một trong những nước đi có tỉ lệ lợi ích trên rủi ro cao nhất khi bí. Điểm cần cẩn thận: đôi khi thứ tự gốc mang thông tin (bài về đoạn liên tiếp), và lúc đó sắp xếp làm mất bài toán.

NÓI LẠI THẬT ĐƠN GIẢN

Hãy tưởng tượng bạn quản lý sổ thu chi của một quán ăn và mỗi ngày có người hỏi “từ ngày 5 tới ngày 20 quán thu bao nhiêu?”. Cách vụng là mỗi lần hỏi lại cộng lại từ đầu. Cách khôn là ngay từ đầu bạn ghi thêm một cột: tổng thu từ đầu tháng tới hết ngày này. Khi có người hỏi, bạn lấy con số ở ngày 20 trừ con số ở ngày 4. Đó là tổng tiền tố, và toàn bộ mẹo chỉ là *trả trước một lần để mọi câu hỏi sau chỉ tốn một phép trừ*.

Ngược lại, nếu sếp nói “cộng thêm 10 nghìn vào mọi ngày từ 5 tới 20” rất nhiều lần trong tháng, thì sửa từng ngày là phí. Bạn chỉ cần ghi “từ ngày 5 tăng thêm 10” và “từ ngày 21 giảm đi 10”, rồi cuối tháng cộng dồn một lượt. Đó là mảng hiệu.

Còn hai con trỏ thì giống hai người cùng đi trên một hàng ghế từ hai đầu, và không ai được quay lui. Vì không ai quay lui nên dù họ đi lâu tới đâu, tổng số bước cũng chỉ bằng chiều dài hàng ghế. Điều kiện để cách này đúng là hàng ghế phải được sắp có trật tự: bước sang phải thì giá trị luôn tăng, bước sang trái thì luôn giảm. Nếu trật tự đó không có — ví dụ trong dãy có số âm làm tổng lúc tăng lúc giảm — thì hai người sẽ bỏ sót chỗ cần tìm, và tệ hơn là chương trình vẫn chạy ngon lành, chỉ đưa ra kết quả sai.

Một câu: *Đừng tính lại từ đầu; hãy cập nhật đúng phần vừa thay đổi.*

Chỗ không rút gọn được: Việc kiểm tra “tính chất này có đơn điệu không” vẫn phải làm bằng lập luận cho từng bài; không có dấu hiệu bề ngoài nào chắc chắn.

Active recall (che cột phải, tự trả lời, rồi mở ra đối chiếu)

Thuật ngữ / câu hỏi	Trả lời
••• Ý chung của cả chương	Khi vòng ngoài tiến một bước, vòng trong đừng quay lại từ đầu — hãy cập nhật phần thay đổi. Biến $O(n^2)$ thành $O(n)$.
••• Tổng tiền tố	$P_i = P_{i-1} + a_i$ với $P_0 = 0$; tổng đoạn $[l, r] = P_r - P_{l-1}$. Luôn để ô đệm P_0 để không có trường hợp riêng.
••• Điều kiện dùng được tổng tiền tố	Phép toán phải có phép ngược: tổng và XOR được, cực đại thì không — cực đại cần sparse table hoặc cây phân đoạn ($B/10$).
••• Mảng hiệu	Cộng v vào $[l, r]$: $D_l += v$, $D_{r+1} -= v$; cuối cùng lấy tổng tiền tố. Dùng khi cập nhật đoạn nhiều, đọc kết quả một lần.
••• Vì sao hai con trỏ tuyến tính	Mỗi chỉ số chỉ đi một chiều và không quay lui, nên tổng quãng đường $\leq 2n$. Độ sâu lồng nhau của code không quyết định chi phí.
••• Điều kiện sống còn của cửa sổ trượt	Tính đơn điệu: đoạn vi phạm thì mọi đoạn chứa nó cũng vi phạm. Mất điều kiện này thì kết quả sai một cách im lặng.
••• Phân ví dụ kinh điển	“Đoạn dài nhất có tổng $\leq S$ ” với mảng có số âm: $[5, -6, 5]$, $S = 4$ — cửa sổ trượt bỏ sót đáp án.
•• Khuôn cửa sổ trượt	Mở rộng r một bước; trong khi vi phạm thì co l ; cập nhật đáp án. Ba dòng, dùng cho hàng trăm bài.
•• Đếm đoạn con có tổng bằng k	$P_r - P_{l-1} = k$: với mỗi r , đếm số P trước đó bằng $P_r - k$ bằng bảng băm. Mẫu “biến điều kiện đoạn thành điều kiện hai điểm nút”.
•• Tổng tiền tố hai chiều	Tổng hình chữ nhật bằng bốn giá trị theo bao hàm–loại trừ; trong thị giác máy tính gọi là ảnh tích phân.
•• Vì sao mảng thẳng con trỏ	Truy cập trực tiếp, dữ liệu liên tiếp thân thiện cache, không cấp phát từng phần tử. Thua khi phải chèn/xóa giữa mảng nhiều.
•• Sắp xếp như nước mở đầu	Nó tạo ra tính đơn điệu — điều kiện đầu vào của hai con trỏ, nhị phân và tham lam. $O(n \log n)$ hầu như luôn trong ngân sách.
• Đặc thù của chuỗi	So sánh chuỗi tốn theo độ dài chứ không $O(1)$; bảng chữ cái nhỏ nên đếm bằng mảng 26 phần tử nhanh hơn bảng băm.

••• phải thuộc •• nên nắm • biết là có

Câu hỏi ôn tập có đáp án (che phần đáp án, tự trả lời trước)

1. Khi nào dùng tổng tiền tố, khi nào phải dùng cây phân đoạn?

| Quyết định bằng hai câu hỏi. Dữ liệu có thay đổi giữa các truy vấn không? Nếu không, tổng tiền tố là đủ và luôn tốt hơn vì hằng số nhỏ hơn nhiều. Nếu có thay đổi, tổng tiền tố phải xây lại $O(n)$ mỗi lần nên hỏng, và bạn cần Fenwick hoặc cây phân đoạn ($B/10$). Câu hỏi thứ hai: phép toán có phép ngược không? Tổng và XOR có nên tổng tiền tố dùng được; cực đại và cực tiểu thì không, phải dùng sparse table cho dữ liệu tĩnh hoặc cây phân đoạn cho dữ liệu động.

2. Làm sao biết chắc một bài giải được bằng cửa sổ trượt trước khi cài?

| Kiểm tính đơn điệu bằng cách hỏi hai câu cụ thể. Một: nếu đoạn $[l, r]$ đã thỏa điều kiện, mọi đoạn con của nó có chắc chắn thỏa không? Hai: nếu đoạn $[l, r]$ vi phạm, mọi đoạn chứa nó có chắc chắn vi phạm không? Cả hai câu đều “có” thì cửa sổ trượt dùng được. Nếu bạn không chắc, hãy thử ngay ba dữ liệu đặc biệt: có số âm, có số 0, và mọi phần tử bằng nhau — ba ca này bắt được phần lớn lỗi đơn điệu.

3. Hai con trỏ và tìm kiếm nhị phân đều cần dữ liệu đã sắp. Chọn cái nào?

| Chọn theo số lượng truy vấn. Nếu bạn cần trả lời một câu hỏi cho mỗi vị trí trái (kiểu “với mỗi l , tìm r lớn nhất thỏa điều kiện”) thì hai con trỏ cho $O(n)$ tổng cộng, tốt hơn $O(n \log n)$ của việc nhị phân từng vị trí. Nhưng hai con trỏ chỉ dùng được khi r là hàm *không giảm* theo l ; nếu không, phải nhị phân. Trong thực tế, viết nhị phân dễ hơn và ít lỗi hơn, nên khi giới hạn cho phép thì đó là lựa chọn an toàn.

4. Tổng tiền tố có bị tràn số không?

| Rất hay bị, và đây là lỗi im lặng kinh điển. Với $n = 10^5$ phần tử, mỗi phần tử tới 10^9 , tổng có thể tới 10^{14} — vượt xa số nguyên 32 bit. Quy tắc: mảng tổng tiền tố gần như luôn phải dùng kiểu 64 bit ngay cả khi mảng gốc là 32 bit. Trong bài lấy modulo, còn phải cẩn thận thêm: hiệu $P_r - P_{l-1}$ có thể âm sau khi lấy dư, nên phải cộng thêm m rồi lấy dư lần nữa.

5. Kỹ thuật này còn dùng ở đâu ngoài bài tập luyện?

| Rất nhiều nơi trong hệ thống thật. Đếm số sự kiện trong một khoảng thời gian trong hệ thống giám sát chính là tổng tiền tố trên trục thời gian. Giới hạn tốc độ theo cửa sổ trượt trong máy chủ web là đúng khuôn ở mục 4. Trong thị giác máy tính, ảnh tích phân dùng để tính nhanh thống kê kê trên vùng ảnh. Trong xử lý dữ liệu cảm biến, trung bình trượt để lọc nhiễu chính là cửa sổ trượt với độ dài cố định.

6. Có mẹo nào tránh lỗi lệch một đơn vị với tổng tiền tố không?

| Ba quy ước cụ thể giúp gần như hết hẳn. Một: luôn để $P_0 = 0$ và đánh số mảng từ 1, khi đó công thức $P_r - P_{l-1}$ đúng cho mọi l kể cả $l = 1$. Hai: viết bất biến ra thành chú thích — “ P_i = tổng của i phần tử đầu tiên” — và mọi chỉ số suy ra từ đó. Ba: kiểm tra bằng một ví dụ ba phần tử đã tính tay trước khi chạy dữ liệu lớn. Đây là ứng dụng trực tiếp của kỷ luật ở A/04.

7. Bài yêu cầu cả cập nhật đoạn lẫn truy vấn đoạn thì sao?

| Không cấu trúc tính nào trong chương này giải được, vì tổng tiền tố mạnh ở truy vấn còn mảng hiệu mạnh ở cập nhật, và ghép trực tiếp thì cái này phá cái kia. Đó chính xác là lý do tồn tại của cây phân đoạn có lazy propagation ở B/10: nó chấp nhận $O(\log n)$ cho *cả hai* phía thay vì $O(1)$ cho một phía và $O(n)$ cho phía kia.

Bài tập (mức: ● khởi động ●● chuẩn ●●● khó)

Bài	Nguồn	Điều cần nhận ra
● Tổng đoạn với truy vấn tĩnh	CSES “Range Sum Queries I”[1]	Tổng tiền tố với ô đệm; chú ý kiểu 64 bit.
● Cập có tổng bằng x trong mảng đã sắp	CSES “Sum of Two Values”[1]	Hai con trỏ; viết bất biến “mọi cặp đã loại đều không thể là đáp án”.

Bài	Nguồn	Điều cần nhận ra (tiếp)
• Tổng đoạn con lớn nhất	CSES “Maximum Subarray Sum”[1]	So sánh cách Kadane với cách tổng tiền tố cộng cực tiểu chạy — hai góc nhìn cho cùng bài.
•• Đoạn con dài nhất không có ký tự lặp	LeetCode “Longest Substring Without Repeating Characters”	Cửa sổ trượt với mảng đếm; kiểm tính đơn điệu trước khi cài.
•• Đếm số đoạn con có tổng bằng x	CSES “Subarray Sums II”[1]	Tổng tiền tố cộng bằng băm; mẫu “điều kiện đoạn thành điều kiện hai điểm nút”.
•• Cập nhật đoạn, truy vấn điểm	CSES “Range Update Queries”[1]	Mảng hiệu; sau đó thử lại bằng Fenwick để thấy khi nào cần cấu trúc mạnh hơn.
•• Tổng hình chữ nhật trong ma trận tĩnh	CSES “Forest Queries”[1]	Tổng tiền tố hai chiều; vẽ hình bao hàm—loại trừ ra giấy trước khi viết công thức.
•• Gộp các đoạn giao nhau	LeetCode “Merge Intervals”	Sắp theo điểm bắt đầu; đây là bài mẫu cho “chọn khoá sắp xếp chính là lời giải”.
••• Cửa sổ nhỏ nhất chứa đủ các ký tự cần thiết	LeetCode “Minimum Window Substring”	Cửa sổ trượt hai chiều với bộ đếm; bài kinh điển để luyện việc co cửa sổ đúng lúc.
••• Nước mưa mắc kẹt	LeetCode “Trapping Rain Water”	Giải cả ba cách (tiền tố cực đại, ngăn xếp B/07, hai con trỏ) và so sánh — bài tập bước 4 tốt nhất của chương.
••• Số đoạn con có đúng k phần tử phân biệt	Bài tập kinh điển	Mẹo “đúng k ” = “nhiều nhất k ” trừ “nhiều nhất $k - 1$ ” — một mẫu biến bài không đơn điệu thành hai bài đơn điệu.
••• Đoạn con dài nhất có tổng $\leq S$ khi mảng có số âm	Bài tập kinh điển	Bài mà cửa sổ trượt sai; cần tổng tiền tố cộng cấu trúc tìm kiếm — hãy tự chỉ ra chính xác chỗ tính đơn điệu gãy.

Kết chương và đọc tiếp

Chương này chỉ có một ý và bốn cách hiện thực hoá nó: đừng tính lại thứ vừa tính. Tổng tiền tố trả trước cho truy vấn; mảng hiệu trả trước cho cập nhật; hai con trỏ và cửa sổ trượt cập nhật gia tăng khi biên di chuyển. Điều kiện dùng được của cả bốn đều quy về tính đơn điệu, và mất tính đơn điệu thì lỗi xảy ra một cách im lặng — nên câu hỏi kiểm tra ở khối *Cần nhớ* đáng thành phần xạ.

Chương tiếp theo mở rộng đúng ý đó cho tình huống mà thứ tự xử lý mới là cái quan trọng: khi bạn cần lấy ra “phần tử đúng lúc” — mới nhất, nhỏ nhất, hay lớn hơn kế tiếp — thay vì quét theo chỉ số. Đó là ngăn xếp, hàng đợi và heap (B/07), và bạn sẽ thấy ngăn xếp đơn điệu là họ hàng gần của hai con trỏ, với cùng lập luận “mỗi phần tử vào ra đúng một lần”.

Nguồn

- [1] A. Laaksonen. “CSES Problem Set.” (2026), address: <https://cses.fi/problemset/> (visited on 09/06/2026).